

Reproducibility Study of LIDA: A Grammar-Agnostic Framework for Generative Visualization

Original Paper: [ACL 2023 Demonstration Track](#)

Arian Yavari
Université Laval
arian.yavari.1@ulaval.ca

April 2026

REPRODUCIBILITY SUMMARY

Scope of Reproducibility This study evaluates the core multi-stage pipeline of the LIDA framework—specifically the Summarizer, Goal Explorer, and VisGenerator modules. We aim to verify the claim that an agentic pipeline can reliably automate data visualization across diverse tabular datasets.

Methodology We conducted a comprehensive re-engineering of the legacy Microsoft/lida repository (v0.0.14) to create a modernized version (v0.1.0). This involved 18,000 insertions and 13,900 deletions to replace the unmaintained llmx dependency with a native lida.llm submodule. The evaluation utilized 23 Vega datasets, tested with MiniMax 2.7 and Gemini 3.1 Pro. The updated repository is available at github.com/arian47/lida.

Results We measured a Visualization Error Rate (VER) of 22.6%, a significant increase from the original paper’s reported 3.5%. While the pipeline logic is verifiable, execution stability is highly sensitive to model-specific code scaffolding and dependency evolution.

Discussion The transition to modern LLM providers reveals a "syntax gap" where alternative models struggle with the original prompt constraints. Successful reproduction required resolving significant technical debt, particularly regarding API modernization and environment containerization.

1 INTRODUCTION

LIDA poses visualization as a multi-stage generation problem. Since its 2023 publication, the software ecosystem has shifted, rendering the original implementation unmaintained. This report documents the technical transformation of the framework and a systematic audit of its reproducibility under 2026 library constraints.

2 TECHNICAL OVERHAUL AND CODE TRANSFORMATION

The transition from version 0.0.14 to 0.1.0 required a near-total restructuring of the core architecture. The primary bottleneck was the `llmx` package, which served as the original LLM interface but has since become obsolete.

To ensure long-term stability, we re-engineered this logic into a native `lida.llm` submodule. This allowed for the removal of Microsoft-specific patterns and the adoption of an MIT license. Architectural improvements included replacing legacy YAML configurations with a centralized Python factory pattern in `factory.py` and rebuilding all data models using Pydantic dataclasses for robust type safety.

3 PROVIDER MODERNIZATION AND DUAL-API STRATEGY

The framework was updated to support a modernized suite of LLM providers. Deprecated support for PaLM and Cohere was removed in favor of Google Gemini and MiniMax 2.7. For the MiniMax integration, we implemented a dual-API strategy utilizing `httpx` directly to resolve path-resolution issues present in standard SDKs. This ensures cross-compatibility between Anthropic and OpenAI-compatible endpoints while maintaining high-speed execution.

4 INFRASTRUCTURE, CONTAINERIZATION, AND TESTING

To resolve the execution inconsistencies noted in the original work, we implemented several infrastructure upgrades:

- **Multi-stage Docker Build:** We optimized layer caching and introduced healthcheck instructions for container orchestration.
- **Pytest Framework:** The original testing suite was entirely rewritten. We implemented comprehensive fixtures for manager instances and text generation configurations, ensuring the Summarizer and VisGenerator can be validated independently.
- **Dependency Management:** The `pyproject.toml` was updated with clearly defined optional extras for web interfaces, infographics, and development environments.

5 METHODOLOGY AND DATASET PREPARATION

We utilized the Vega datasets repository as our benchmark. Following a strict compatibility audit, 23 CSV-based datasets were retained. Hierarchical JSON files and non-tabular formats were excluded to maintain the focus on tabular processing pipelines.

To ensure maximum determinism, the LLM Temperature was set to 0. For each of the 23 datasets, the pipeline was tasked with generating one summary and five visualization goals, resulting in 115 total code-generation attempts.

6 EVALUATION RESULTS

Comprehensive execution logs, including the raw generated code for every attempt, are documented in **Appendix 1**, which contains the original evaluation Jupyter Notebook exported in PDF format.

6.1 Quantitative Analysis: Visualization Error Rate (VER)

The VER is defined as the ratio of failed code executions to the total number of attempts:

$$VER = \left(\frac{V_{err}}{N} \right) \times 100 = \left(\frac{26}{115} \right) \times 100 \approx 22.6\% \quad (1)$$

The original paper reported a VER of 3.5%. Our higher error rate is largely attributable to "syntax drift" and the increased strictness of modern Python visualization libraries compared to the environment used during the original 2023 study.

6.2 Qualitative Analysis: SEVQ Metric

Successful visualizations were judged by Gemini 3.1 Pro using the Self-Evaluated Visualization Quality metric. We assessed Accuracy, Transformation, Goal Compliance, Type, Encoding, and Aesthetics. Detailed rationales for these scores are provided in the supplementary evaluation files.

7 DISCUSSION OF FAILURE MODES

Detailed analysis of the 26 failed instances revealed six primary challenges:

- **Parsing Artifacts:** The inclusion of Markdown backticks or redundant chain-of-thought commentary within the code stream.
- **Scaffolding Leakage:** Retention of template tags such as <stub> or <imports> in the final Python output.
- **Syntax and Indentation:** Significant inconsistencies in code structure that prevented the Python interpreter from executing the logic.
- **Semantic Hallucination:** Generating goals for non-existent columns or creating imaginary data outliers.
- **Dependency Deprecation:** Attempts to import obsolete modules such as `mpl_toolkits.basemap`.
- **Aesthetic Collision:** Overlapping legends and axis labels that rendered the charts unreadable.

8 CONCLUSION

While LIDA remains a robust conceptual blueprint for generative data science, its practical reproducibility is fragile. This study demonstrates that the grammar-agnostic promise of the framework requires constant maintenance of the code-generation scaffolding. The successful reproduction of the pipeline was only possible through a massive structural overhaul of the original LLM interface and dependency logic.

References

- [1] Victor Dibia. 2023. *LIDA: A Tool for Automatic Generation of Grammar-Agnostic Visualizations and Infographics using Large Language Models*. Proceedings of the 61st ACL (System Demonstrations). <https://aclanthology.org/2023.acl-demo.11/>

Supplementary Material: For the full distribution of evaluation code and qualitative scoring, please refer to: Appendix 1 (Evaluation_Results.pdf), which contains the exported Jupyter Notebook outputs.